# OBSERVACIONES DE LA PRÁCTICA

Gabriel Andrés Moya Caravaca - 202616964
Edgar Daniel Gonzalez Martinez - 202524741
Gerardo Rocha Linares - 202513388

| | Máquina 1 |
|---|---|
| **Procesador** | AMD Fyzen 5 4500U |
| **Memoria RAM (GB)** | 16GB Ram |
| **Sistema Operativo** | Windows 10 |

*Tabla 1. Especificaciones de la máquina para ejecutar las pruebas de rendimiento.*

## Resultados

### ***Carga de Catálogo LINEAR PROBING***

| Factor de Carga (PROBING) | Consumo de Datos [kB] | Tiempo de Ejecución Real @LP [ms] |
|---|---|---|
| 0.1 | 11611130.90 | 239854.04 |
| 0.5 | 2600457.90 | 52952.84 |
| 0.7 | 1962774.39 | 48204.05 |
| 0.9 | 1611279.97 | 46864.65 |

*Tabla 2. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando LINEAR PROBING*

### ***Carga de Catálogo SEPARATE CHAINING***

| Factor de Carga (CHAINING) | Consumo de Datos [kB] | Tiempo de Ejecución Real @SC [ms] |
|---|---|---|
| 2.00 | 951270.25 | 39366.30 |
| 4.00 | 668048.81 | 33265.89 |
| 6.00 | 573268.12 | 32869.12 |
| 8.00 | 528213.02 | 34370.24 |

*Tabla 3. Comparación de consumo de datos y tiempo de ejecución para carga de catálogo con el índice por categorías utilizando SEPARATE CHAINING*

### Gráficas

La gráfica generada por los resultados de las pruebas de rendimiento**.**

- Comparación de memoria y tiempo de ejecución para LINEAR PROBING y SEPARATE CHAINING
- Comparación de factor de carga y memoria para LINEAR PROBING y SEPARATE CHAINING

## Preguntas de análisis

1. ¿Por qué en la función getTime() se utiliza time.perf_counter() en vez de otras funciones como time.process_time()?
   Se debe a la función que cumple cada una. Por un lado, time.perf_counter() mide el tiempo real de ejecución del programa, incluyendo posibles pausas o esperas. Por otro lado, time.process_time() mide únicamente el tiempo de CPU utilizado, sin tener en cuenta esas pausas. Por esta razón, perf_counter() es más adecuado para medir el tiempo total de ejecución.

2. ¿Por qué son importantes las funciones start() y stop() de la librería tracemalloc?
   Las funciones **start()** y **stop()** de tracemalloc en python permiten iniciar y detener el rastreo de memoria, lo que hace posible medir de forma precisa el consumo de memoria de partes específicas del código.

3. ¿Por qué no se puede medir paralelamente el uso de memoria y el tiempo de ejecución de las operaciones?
   medir el tiempo o el uso de memoria aumenta la complejidad en la ejecución del programa. Por ejemplo, herramientas como tracemalloc deben registrar asignaciones de memoria, lo que consume tiempo adicional, y las funciones de medición de tiempo también añaden pequeñas operaciones extra. Esta sobrecarga afeta directamente el rendimiento del programa, alterando tanto el tiempo de ejecución como el consumo de memoria, por lo que medir ambas cosas simultáneamente puede generar resultados menos precisos.

4. Dado el número de elementos de los archivos (large), ¿Cuál sería el factor de carga para estos índices según su mecanismo de colisión?
   En el archivo *books.csv* se almacenan 10.000 libros diferentes, lo que influye directamente en la elección del mecanismo de colisiones. En el caso de *linear probing*, se requiere un factor de carga bajo (aproximadamente 0.5) para evitar el rehash, por lo que el tamaño del arreglo debe ser cercano al doble del número de elementos. A partir de la fórmula $\alpha=n/m$, se tiene que $0.5=10.000/M$ *despejando M se obtiene* $M=10.000/0.5$ lo que da un tamaño de tabla de 20.000, y ajustando al siguiente número primo se obtiene 20.011, con un factor de carga real de aproximadamente 0.4997. Por otro lado, en *separate chaining* el factor de carga puede ser 4 y representa el número promedio de elementos por bucket. Si se desea un promedio de 4 elementos por posición, se tiene $4=10,000/M$ *despejando M se obtiene* $M=10.000/4$, lo que da un tamaño de tabla de 2.500, manteniendo un factor de carga de 4.

5. ¿Qué cambios percibe en el tiempo de ejecución al modificar el factor de carga máximo?
   Si se modifica el factor de carga máximo
   Al modificar el factor de carga máximo se generan cambios en el tiempo de ejecución debido al aumento o disminución de colisiones en la tabla hash. Cuando el factor de carga es alto, las colisiones aumentan y las operaciones como búsqueda e inserción se vuelven más lentas. Cuando es bajo, las operaciones son más rápidas pero se utiliza más memoria. Adicionalmente la distribución de datos empeoraría lo que generaría agrupamientos indeseados en la tabla.

6. ¿Qué cambios percibe en el consumo de memoria al modificar el factor de carga máximo?
   Al modificar el factor de carga máximo se observan cambios en el consumo de memoria debido al tamaño de la tabla hash. Cuando el factor de carga es bajo, se necesita una tabla más grande para almacenar la misma cantidad de elementos, lo que incrementa el uso de memoria. En cambio, cuando el factor de carga es alto, la tabla puede ser más pequeña, reduciendo el consumo de memoria, aunque a costa de más colisiones.

7. ¿Qué cambios percibe en el tiempo de ejecución al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.
   Al modificar el esquema de colisiones se perciben cambios en el tiempo de ejecución debido a la forma en que se manejan las colisiones. En linear probing, las colisiones se resuelven buscando posiciones libres en el arreglo, lo que puede generar agrupamientos y aumentar significativamente el tiempo de búsqueda e inserción a medida que crece el número de colisiones. En cambio, en separate chaining, las colisiones se manejan mediante listas en cada posición, por lo que el acceso depende de la longitud de estas listas.

8. ¿Qué cambios percibe en el consumo de memoria al modificar el esquema de colisiones? Si los percibe, describa las diferencias y argumente su respuesta.
   Linear probing usa más memoria en la tabla (por espacios vacíos), mientras que separate chaining usa menos espacio en la tabla pero más memoria por las listas adicionales.